NAME:- Vatsal Umrao

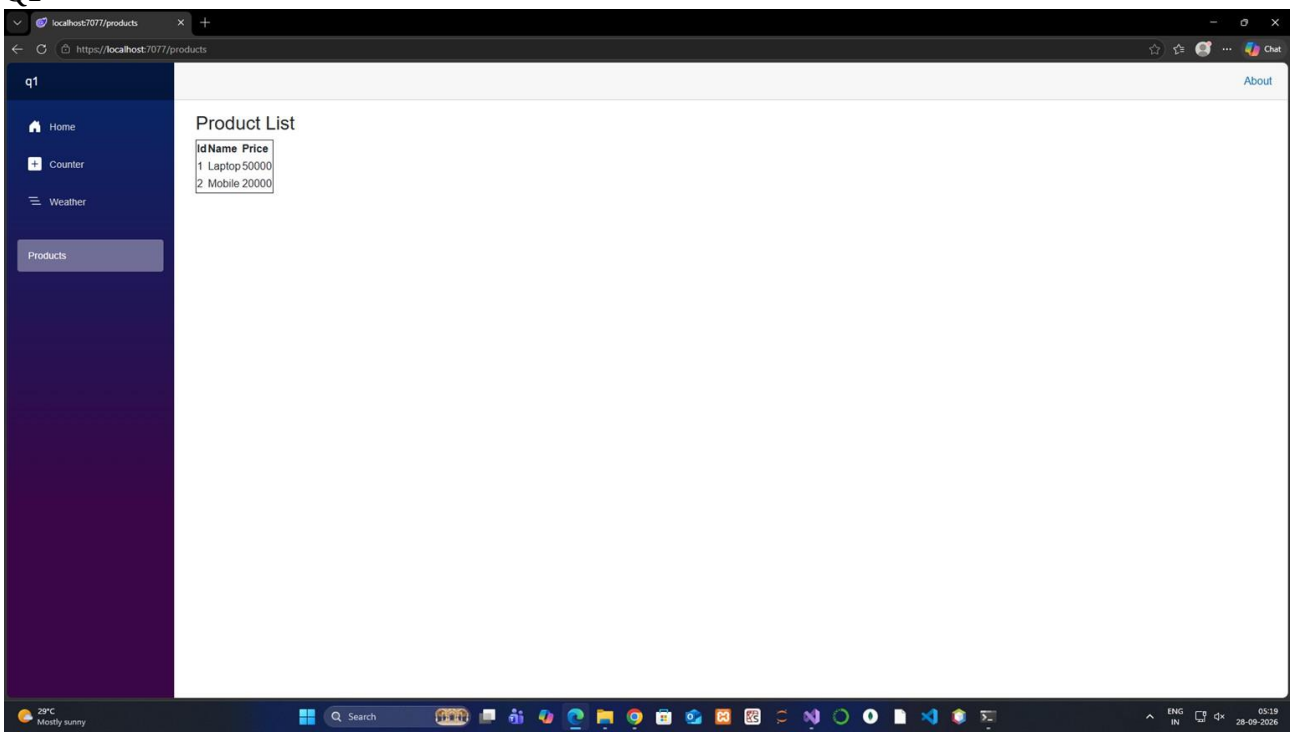
DIV:- B

ROLLNO:92

SUBJECT:- .NET/702

ASSIGNMENT--2

Q1




```
@page "/products"

<h3>Product List</h3>

<table border="1">
    <tr>
        <th>Id</th>
```

```
        <th>Name</th>
        <th>Price</th>
    </tr>

    @foreach (var p in products)
    {
        <tr>
            <td>@p.Id</td>
            <td>@p.Name</td>
            <td>@p.Price</td>
        </tr>
    }
</table>

@code {
    List<Product> products = new()
    {
        new Product{Id=1,Name="Laptop",Price=50000},
        new Product{Id=2,Name="Mobile",Price=20000}
    };
}

public class Product
{
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
}
```

Q-2

```
@page "/students"

<h1>Student Dashboard</h1>

<!-- WITHOUT PARAM -->
<StudentWithoutParam />

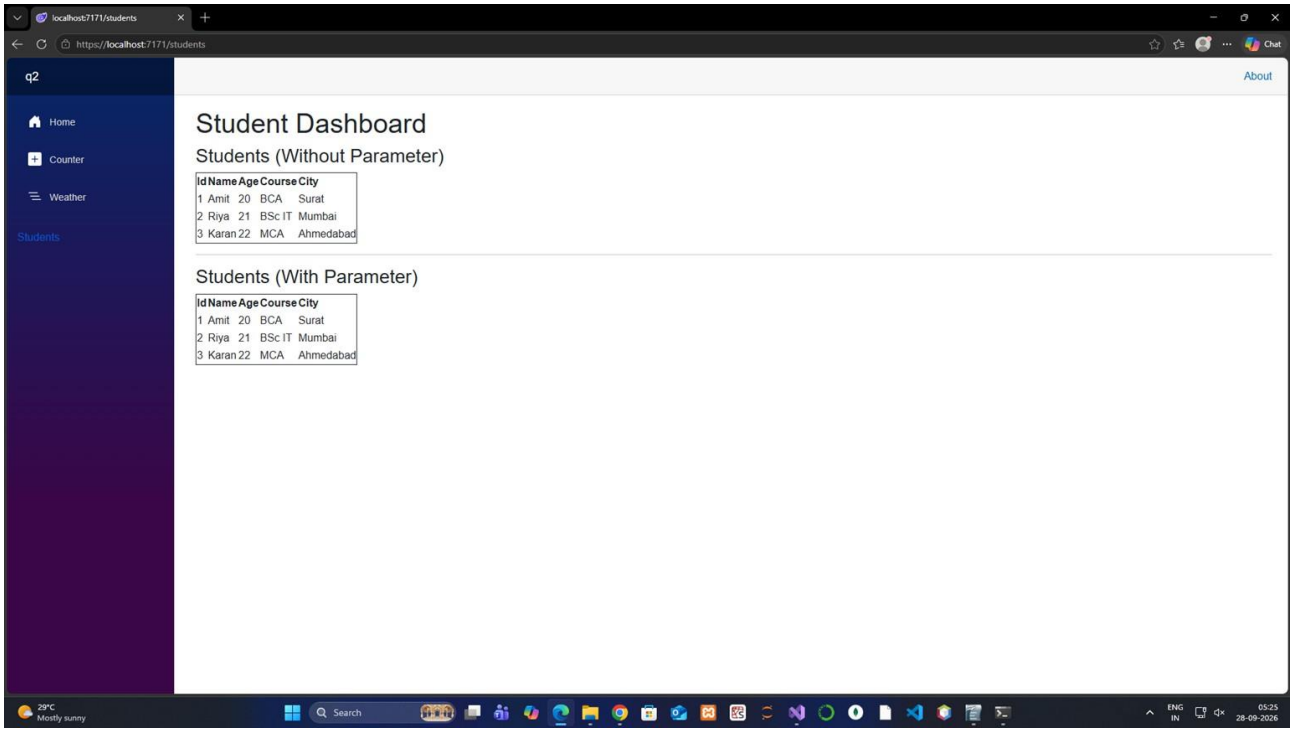
<hr />

<!-- WITH PARAM -->
<StudentWithParam Students="@studentList" />

@code {
    List<Student> studentList = new()
    {
        new Student{Id=1,Name="Amit",Age=20,Course="BCA",City="Surat"},
        new Student{Id=2,Name="Riya",Age=21,Course="BSc IT",City="Mumbai"},
        new Student{Id=3,Name="Karan",Age=22,Course="MCA",City="Ahmedabad"}
    };
```

```
}


public class Student
{
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
        public string City { get; set; }
}
```



q-3

```
@page "/dashboard"

<h1>Dashboard</h1>

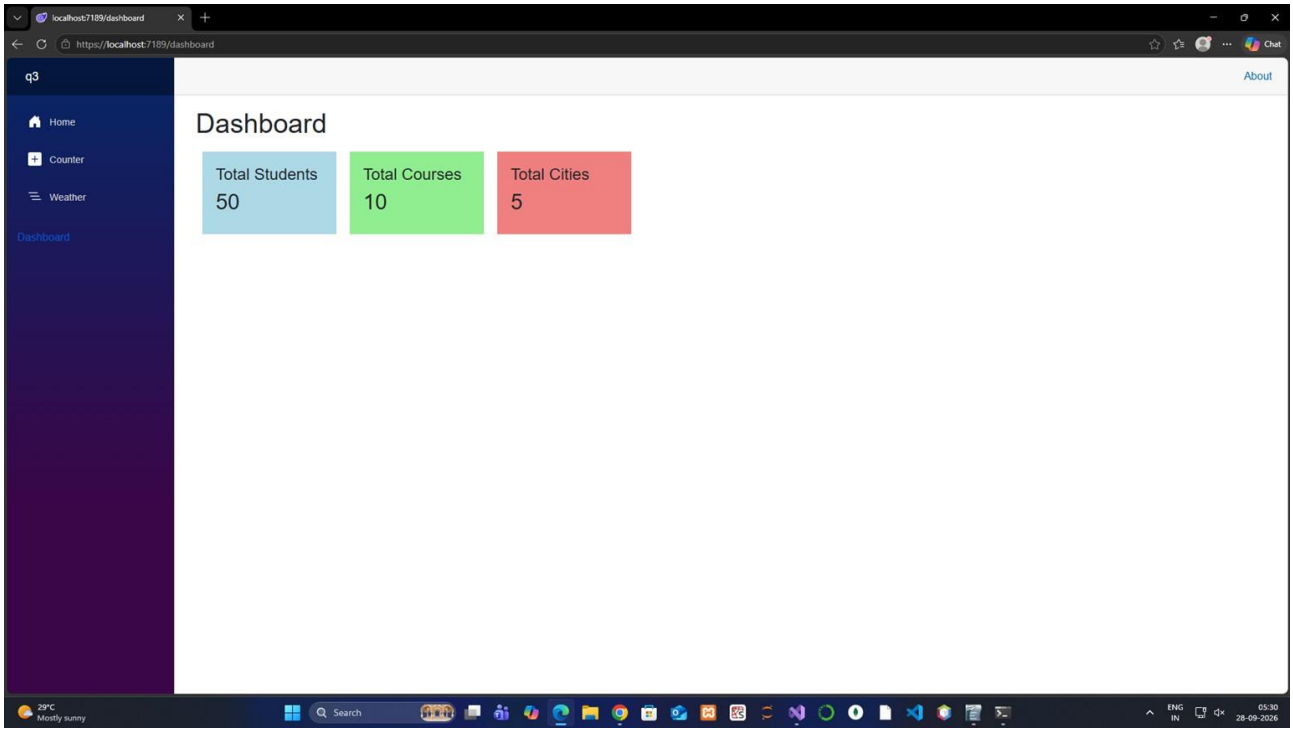
<div style="display:flex">
   <TotalStudents />
   <TotalCourses />
   <TotalCities />
</div>

<div style="background:lightcoral;padding:20px;margin:10px;width:200px">
   <h4>Total Cities</h4>
   <h2>5</h2>
</div>

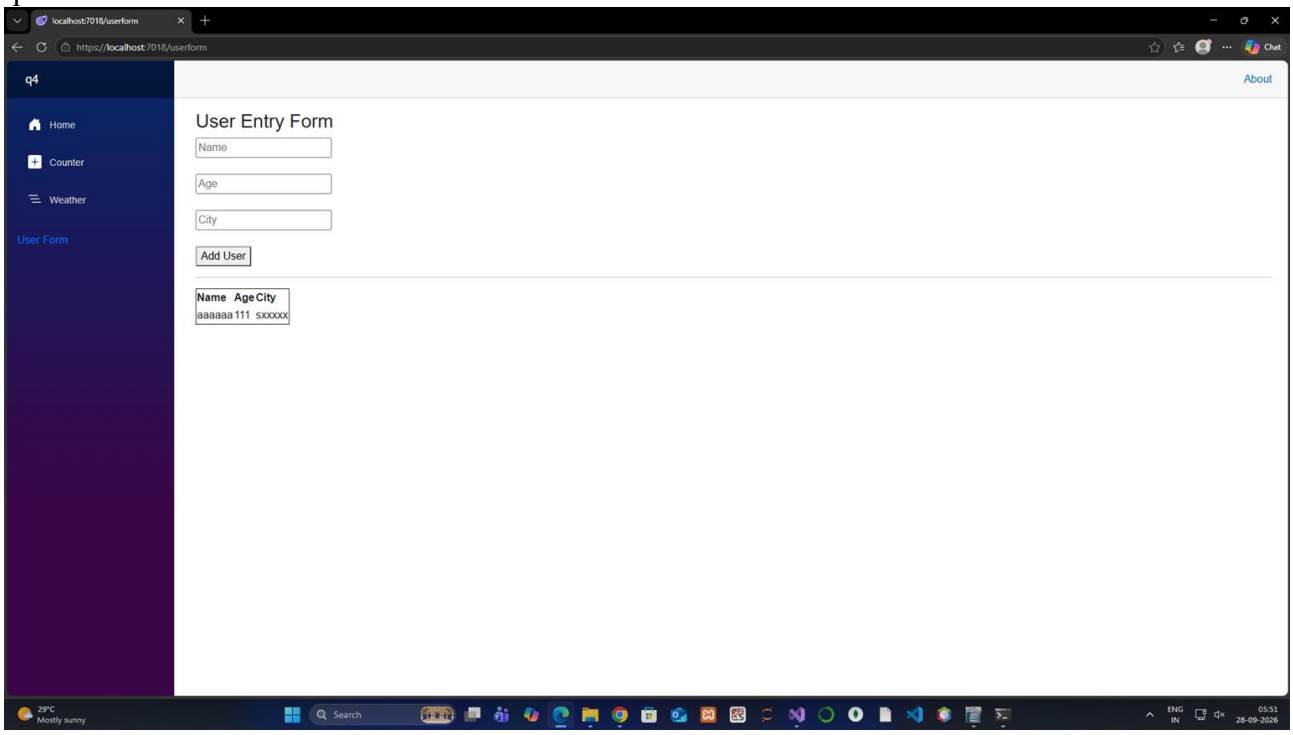
<div style="background:lightgreen;padding:20px;margin:10px;width:200px">
   <h4>Total Courses</h4>
   <h2>10</h2>
```

```
</div>

<div style="background:lightblue;padding:20px;margin:10px;width:200px">
    <h4>Total Students</h4>
    <h2>50</h2>
</div>
```



q-4




```
@page "/userform"
```

```razor
@rendermode InteractiveServer
@using q4.Models

<h3>User Entry Form</h3>

<input @bind="Name" placeholder="Name" />
<br />
<br />

<input @bind="Age" placeholder="Age" />
<br />
<br />

<input @bind="City" placeholder="City" />
<br />
<br />

<button @onclick="AddUser">Add User</button>

<hr />

@if (Users.Count == 0)
{
    <p>No data</p>
}
else
{
    <table border="1">
        <tr>
            <th>Name</th>
            <th>Age</th>
            <th>City</th>
        </tr>

        @foreach (var u in Users)
        {
            <tr>
                <td>@u.Name</td>
                <td>@u.Age</td>
                <td>@u.City</td>
            </tr>
        }
    </table>
}

@code {
    string Name = "";
    string Age = "";
    string City = "";

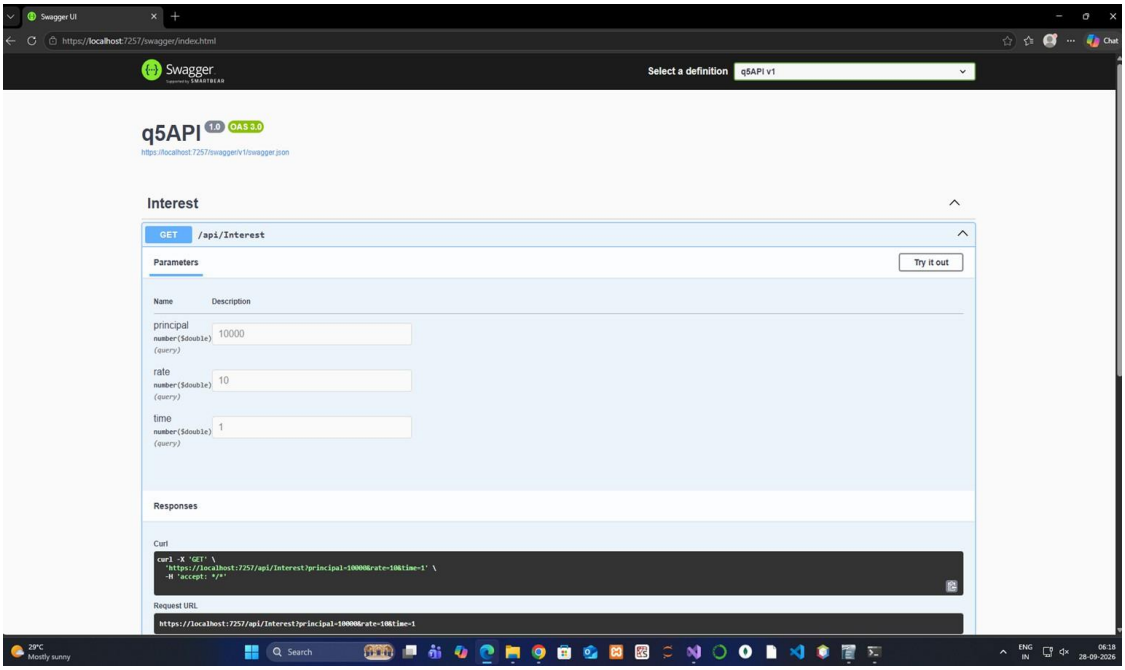
    List<User> Users = new();

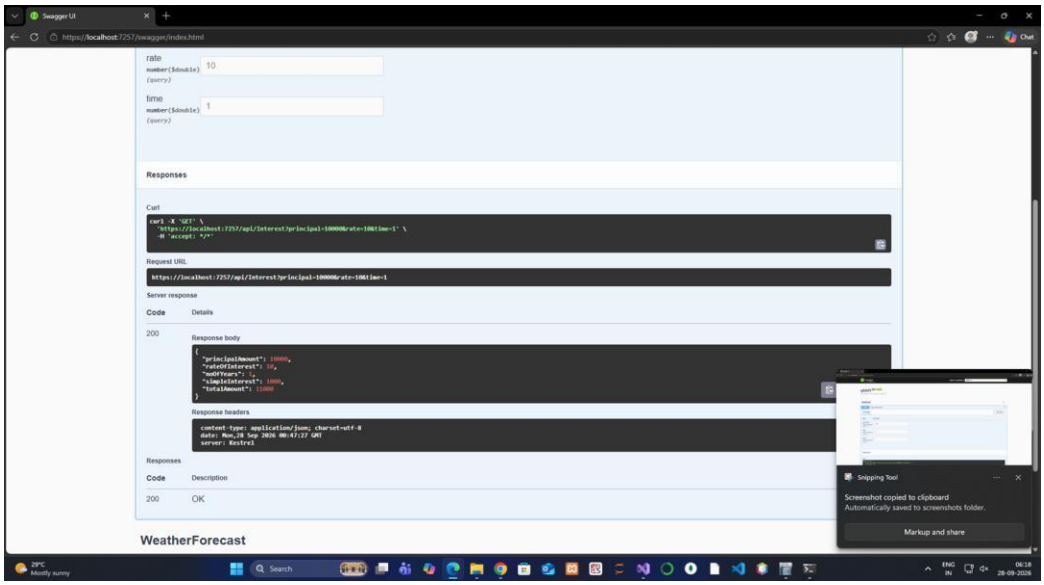
    void AddUser()
    {
        Users.Add(new User
        {
            Name = Name,
            Age = int.TryParse(Age, out var a) ? a : 0,
            City = City
        });

        Name = "";
        Age = "";
        City = "";
    }
```

```
}
namespace q4.Models
{
	public class User
	{
		public string Name { get; set; }
		public int Age { get; set; }
		public string City { get; set; }
	}
}
```

q-5



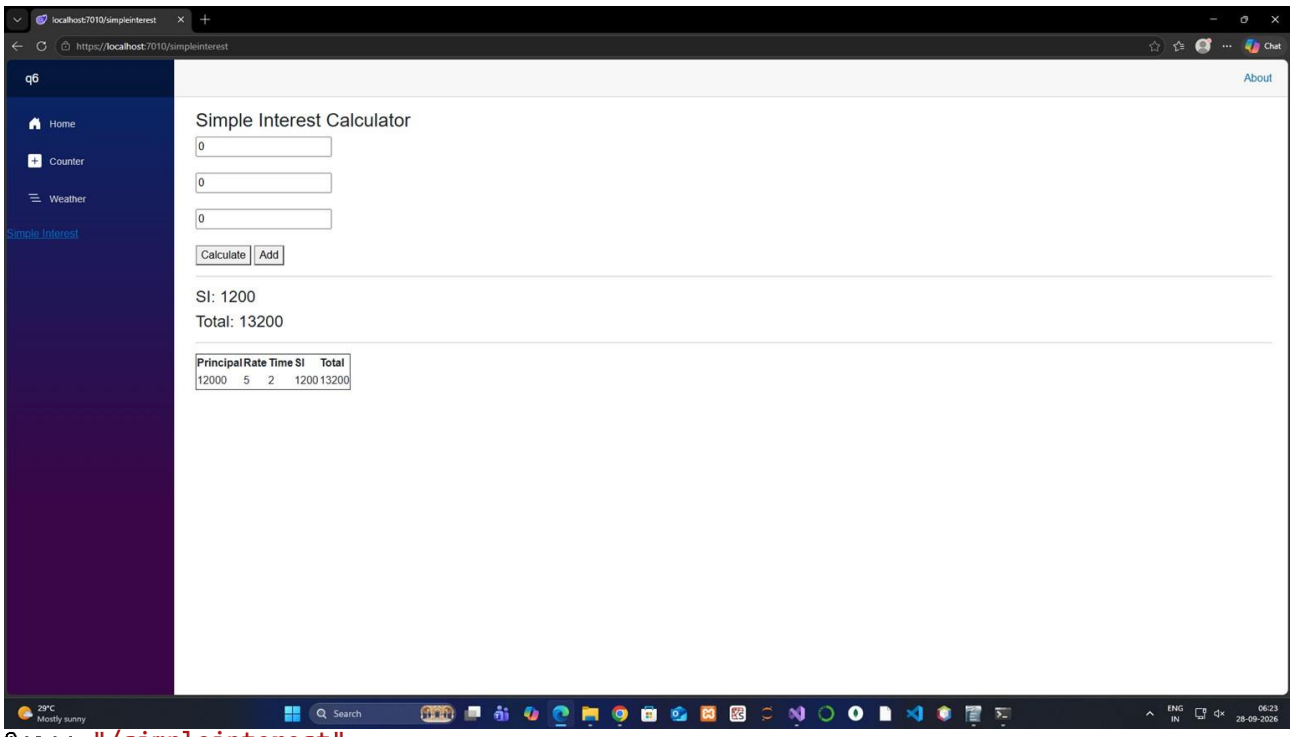


```
using Microsoft.AspNetCore.Mvc;

namespace SimpleInterestAPI.Controllers
{
	[ApiController]
	[Route("api/[controller]")]
	public class InterestController : ControllerBase
	{
		[HttpGet]
		public IActionResult Get(double principal, double rate, double time)
		{
			double si = (principal * rate * time) / 100;
			double total = principal + si;

			return Ok(new
			{
				PrincipalAmount = principal,
				RateOfInterest = rate,
				NoOfYears = time,
				SimpleInterest = si,
				TotalAmount = total
			});
		}
	}
}
```

q-6




```
@page "/simpleinterest"
@rendermode InteractiveServer

<h3>Simple Interest Calculator</h3>

<input @bind="principal" placeholder="Principal" />
<br />
<br />

<input @bind="rate" placeholder="Rate" />
<br />
<br />

<input @bind="time" placeholder="Time" />
<br />
<br />

<button @onclick="Calculate">Calculate</button>
<button @onclick="AddRecord">Add</button>

<hr />

<h4>SI: @si</h4>
<h4>Total: @total</h4>

<hr />

<table border="1">
    <tr>
        <th>Principal</th>
        <th>Rate</th>
        <th>Time</th>
        <th>SI</th>
        <th>Total</th>
    </tr>

    @foreach (var r in records)
    {
        <tr>
            <td>@r.Principal</td>
            <td>@r.Rate</td>
```

```
            <td>@r.Time</td>
            <td>@r.SI</td>
            <td>@r.Total</td>
        </tr>
    }
</table>

@code {
    double principal;
    double rate;
    double time;

    double si;
    double total;

    List<Record> records = new();

    void Calculate()
    {
        si = (principal * rate * time) / 100;
        total = principal + si;
    }

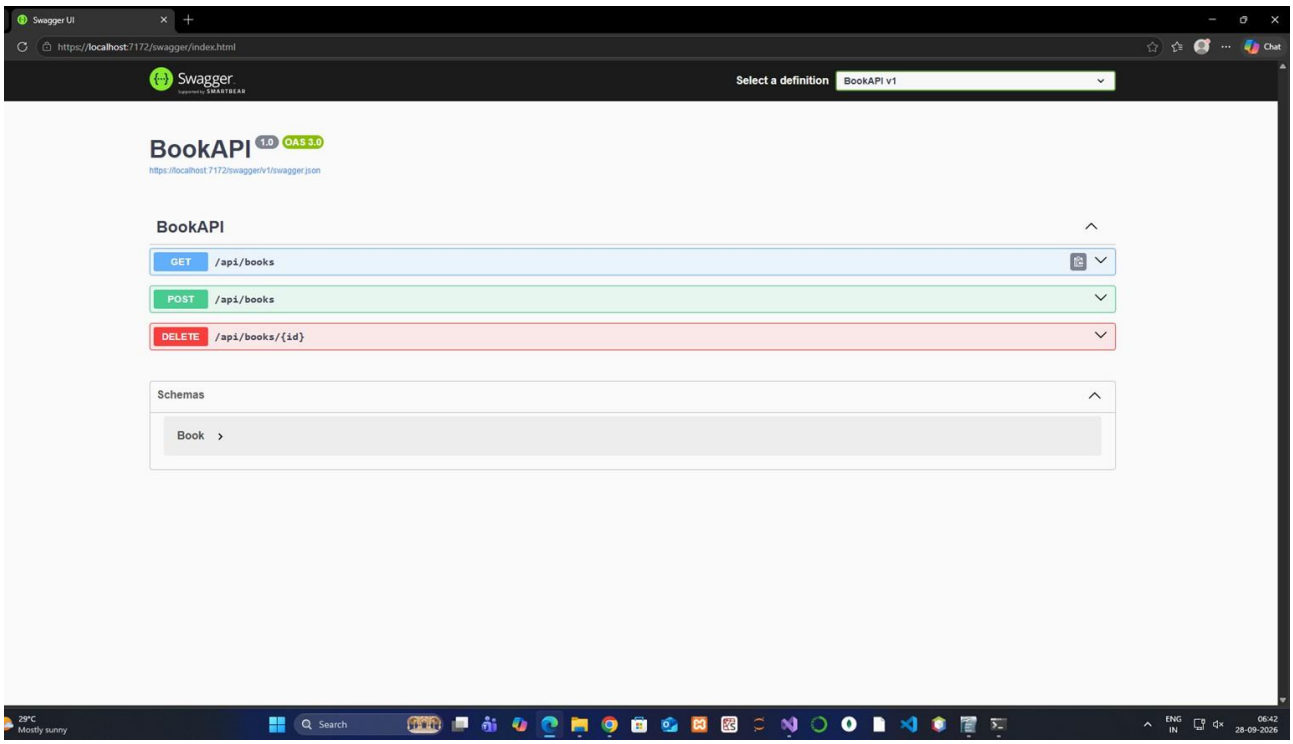
    void AddRecord()
    {
        Calculate();

        records.Add(new Record
        {
            Principal = principal,
            Rate = rate,
            Time = time,
            SI = si,
            Total = total
        });

        principal = 0;
        rate = 0;
        time = 0;
    }

    public class Record
    {
        public double Principal { get; set; }
        public double Rate { get; set; }
        public double Time { get; set; }
        public double SI { get; set; }
        public double Total { get; set; }
    }
}
```

q-7




```
namespace BookAPI.Models
{
	public class Book
	{
		public int Id { get; set; }
		public string Title { get; set; }
		public string Author { get; set; }
		public double Price { get; set; }
	}
}


var builder = WebApplication.CreateBuilder(args);

// 👇 ADD SWAGGER SERVICES
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// 👇 ENABLE SWAGGER UI
app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();

List<Book> books = new List<Book>();

app.MapGet("/api/books", () => books);

app.MapPost("/api/books", (Book book) =>
{
	books.Add(book);
	return book;
});

app.MapDelete("/api/books/{id}", (int id) =>
{
	var b = books.FirstOrDefault(x => x.Id == id);
```

```
        if (b != null) books.Remove(b);
        return books;
});

app.Run();

public class Book
{
        public int Id { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public double Price { get; set; }
}

@page "/books"
@rendermode InteractiveServer
@using System.Net.Http.Json

<h3>Book CRUD</h3>

<input placeholder="Id" @bind="book.Id" />
<br />
<input placeholder="Title" @bind="book.Title" />
<br />
<input placeholder="Author" @bind="book.Author" />
<br />
<input placeholder="Price" @bind="book.Price" />
<br />
<br />

<button @onclick="AddBook">Add</button>
<button @onclick="LoadBooks">Refresh</button>

<hr />

<table border="1">
    <tr>
        <th>Id</th>
        <th>Title</th>
        <th>Author</th>
        <th>Price</th>
        <th>Action</th>
    </tr>

    @foreach (var b in books)
    {
        <tr>
            <td>@b.Id</td>
            <td>@b.Title</td>
            <td>@b.Author</td>
            <td>@b.Price</td>
            <td>
                <button @onclick="() => DeleteBook(b.Id)">Delete</button>
            </td>
        </tr>
    }
</table>

@code {

    Book book = new();
    List<Book> books = new();

    HttpClient http = new HttpClient();
```

```
    protected override async Task OnInitializedAsync()
    {
        await LoadBooks();
    }

    async Task LoadBooks()
    {
        books = await
http.GetFromJsonAsync<List<Book>>("https://localhost:7172/api/books");
    }

    async Task AddBook()
    {
        await http.PostAsJsonAsync("https://localhost:7172/api/books", book);
        book = new Book();
        await LoadBooks();
    }

    async Task DeleteBook(int id)
    {
        await http.DeleteAsync($"https://localhost:7172/api/books/{id}");
        await LoadBooks();
    }

    public class Book
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public double Price { get; set; }
    }
}

using BookUI.Components;


namespace BookUI
{
      public class Program
      {
            public static void Main(string[] args)
            {
                  var builder = WebApplication.CreateBuilder(args);

                  // Add services to the container.
                  builder.Services.AddRazorComponents()
                        .AddInteractiveServerComponents();

                  var app = builder.Build();

                  // Configure the HTTP request pipeline.
                  if (!app.Environment.IsDevelopment())
                  {
                        app.UseExceptionHandler("/Error");
                        // The default HSTS value is 30 days. You may want to change
this for production scenarios, see https://aka.ms/aspnetcore-hsts.
                        app.UseHsts();
                  }

                  app.UseHttpsRedirection();

                  app.UseStaticFiles();
                  app.UseAntiforgery();

                  app.MapRazorComponents<App>()
```

```
                        .AddInteractiveServerRenderMode();

                    app.Run();
                }
            }
}
<div class="top-row ps-3 navbar navbar-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="">BookUI</a>
    </div>
</div>

<input type="checkbox" title="Navigation menu" class="navbar-toggler" />

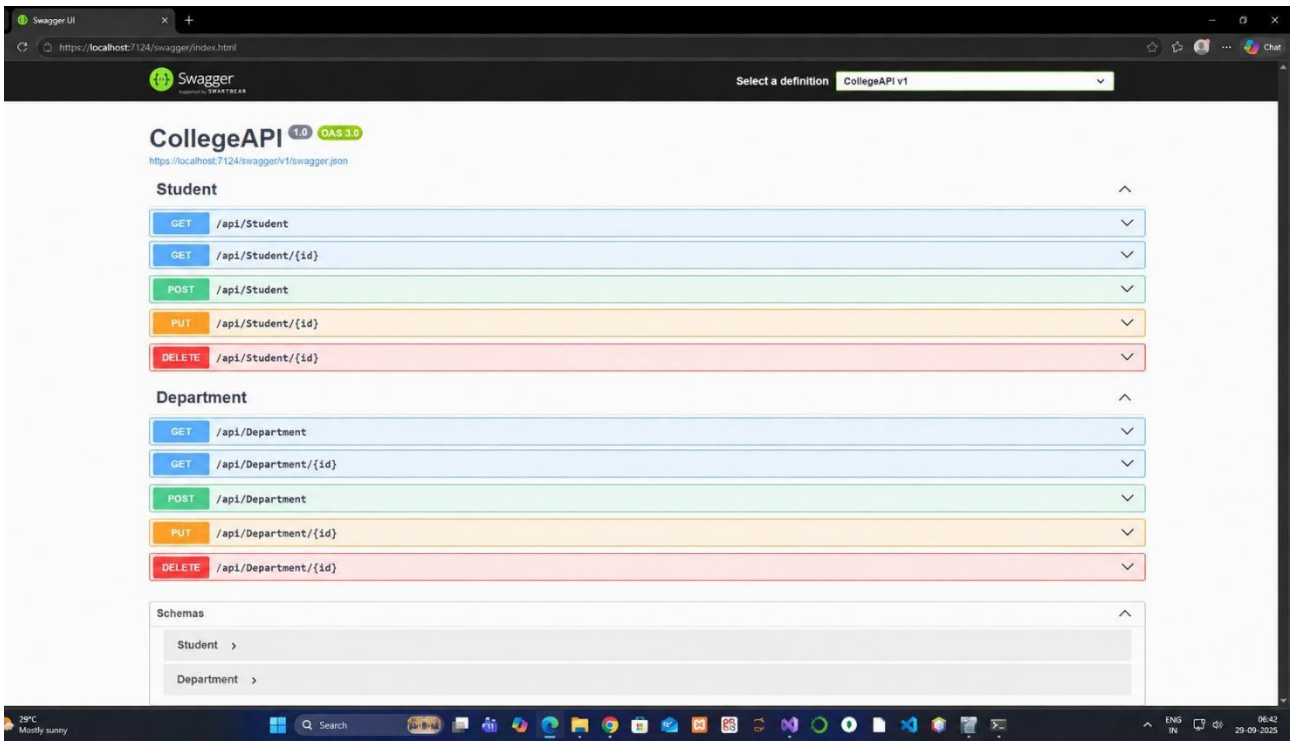
<div class="nav-scrollable" onclick="document.querySelector('.navbar-
toggler').click()">
    <nav class="flex-column">
        <div class="nav-item px-3">
            <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
                <span class="bi bi-house-door-fill-nav-menu" aria-
hidden="true"></span> Home
            </NavLink>
        </div>

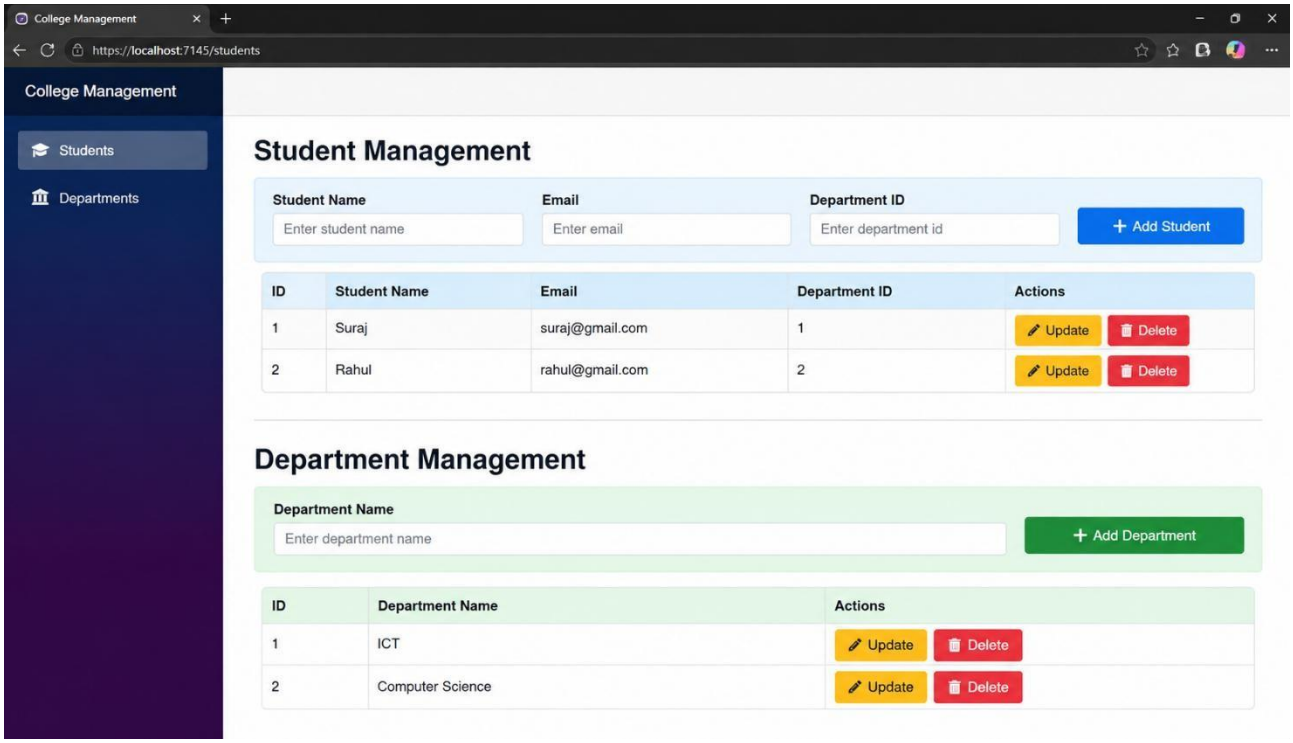
        <div class="nav-item px-3">
            <NavLink class="nav-link" href="counter">
                <span class="bi bi-plus-square-fill-nav-menu" aria-
hidden="true"></span> Counter
            </NavLink>
        </div>

        <div class="nav-item px-3">
            <NavLink class="nav-link" href="weather">
                <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span>
Weather
            </NavLink>
        </div>

        <NavLink href="books">
            Book CRUD
        </NavLink>
    </nav>
</div>
```

Q-8:-






```
CREATE DATABASE CollegeDB;
GO

USE CollegeDB;
GO

CREATE TABLE Department
(
```

```
    DepartmentId INT PRIMARY KEY IDENTITY(1,1),
    DepartmentName VARCHAR(100) NOT NULL
);
GO

CREATE TABLE Student
(
    StudentId INT PRIMARY KEY IDENTITY(1,1),
    StudentName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    DepartmentId INT,

    FOREIGN KEY (DepartmentId)
        REFERENCES Department(DepartmentId)
);
GO

INSERT INTO Department (DepartmentName)
VALUES
('ICT'),
('Computer Science');
GO

INSERT INTO Student
(StudentName, Email, DepartmentId)
VALUES
('Suraj', 'suraj@gmail.com', 1),
('Rahul', 'rahul@gmail.com', 2);
GO


using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<CollegeDbContext>(options =>
    options.UseSqlServer(
        "Server=(localdb)\\MSSQLLocalDB;" +
        "Database=CollegeDB;" +
        "Trusted_Connection=True;" +
        "TrustServerCertificate=True"));

builder.Services.AddCors(options  =>
{
    options.AddPolicy("AllowBlazor", policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
```

```
});

var app = builder.Build();

app.UseHttpsRedirection();

app.UseCors("AllowBlazor");

app.MapControllers();

app.Run();


// ================= DATABASE CONTEXT =================

public class CollegeDbContext : DbContext
{
    public CollegeDbContext(
        DbContextOptions<CollegeDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }

    public DbSet<Department> Departments { get; set; }
}


// ================= STUDENT MODEL =================

public class Student
{
    public int StudentId { get; set; }

    public string StudentName { get; set; } = "";

    public string Email { get; set; } = "";

    public int DepartmentId { get; set; }
}


// ================= DEPARTMENT MODEL ================

public class Department
{
    public int DepartmentId { get; set; }

    public string DepartmentName { get; set; } = "";
}
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

[Route("api/[controller]")]
[ApiController]
public class StudentController : ControllerBase
{
    private readonly CollegeDbContext db;

    public StudentController(CollegeDbContext db)
    {
        this.db = db;
    }

    // GET
    [HttpGet]
    public async Task<IActionResult> GetStudents()
    {
        return Ok(await db.Students.ToListAsync());
    }

    // GET BY ID
    [HttpGet("{id}")]
    public async Task<IActionResult> GetStudent(int id)
    {
        var student = await db.Students.FindAsync(id);

        if (student == null)
            return NotFound();

        return Ok(student);
    }

    // POST
    [HttpPost]
    public async Task<IActionResult> AddStudent(Student student)
    {
        db.Students.Add(student);

        await db.SaveChangesAsync();

        return Ok(student);
    }

    // PUT
    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateStudent(
        int id,
        Student student)
```

```
    {
        if (id != student.StudentId)
            return BadRequest();

        db.Entry(student).State =
            EntityState.Modified;

        await db.SaveChangesAsync();

        return Ok(student);
    }

    // DELETE
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteStudent(int id)
    {
        var student =
            await db.Students.FindAsync(id);

        if (student == null)
            return NotFound();

        db.Students.Remove(student);

        await db.SaveChangesAsync();

        return Ok("Student deleted successfully");
    }
}
\


using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

[Route("api/[controller]")]
[ApiController]
public class DepartmentController : ControllerBase
{
    private readonly CollegeDbContext db;

    public DepartmentController(CollegeDbContext db)
    {
        this.db = db;
    }

    // GET
    [HttpGet]
    public async Task<IActionResult> GetDepartments()
    {
        return Ok(await db.Departments.ToListAsync());
    }
```

```
// GET BY ID
[HttpGet("{id}")]
public async Task<IActionResult> GetDepartment(int id)
{
    var department =
        await db.Departments.FindAsync(id);

    if (department == null)
        return NotFound();

    return Ok(department);
}

// POST
[HttpPost]
public async Task<IActionResult> AddDepartment(
    Department department)
{
    db.Departments.Add(department);

    await db.SaveChangesAsync();

    return Ok(department);
}

// PUT
[HttpPut("{id}")]
public async Task<IActionResult> UpdateDepartment(
    int id,
    Department department)
{
    if (id != department.DepartmentId)
        return BadRequest();

    db.Entry(department).State =
        EntityState.Modified;

    await db.SaveChangesAsync();

    return Ok(department);
}

// DELETE
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteDepartment(int id)
{
    var department =
        await db.Departments.FindAsync(id);

    if (department == null)
        return NotFound();
```

```
        db.Departments.Remove(department);

        await db.SaveChangesAsync();

        return Ok("Department deleted successfully");
    }
}
```

```
using CollegeBlazor.Components;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

builder.Services.AddHttpClient("CollegeAPI", client =>
{
    client.BaseAddress =
        new Uri("https://localhost:7124/");
});

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseAntiforgery();

app.MapStaticAssets();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run();
```

```
@page "/students"
@rendermode InteractiveServer

@using System.Net.Http.Json
@inject IHttpClientFactory ClientFactory
```

```
<h3>Student Management</h3>

<input @bind="student.StudentName"
       placeholder="Student Name" />

<input @bind="student.Email"
       placeholder="Email" />

<input @bind="student.DepartmentId"
       type="number"
       placeholder="Department ID" />

<button @onclick="AddStudent">
   Add Student
</button>

<hr />

<table class="table">

   <thead>
      <tr>
         <th>ID</th>
         <th>Name</th>
         <th>Email</th>
         <th>Department ID</th>
         <th>Action</th>
      </tr>
   </thead>

   <tbody>

      @foreach (var s in students)
      {
         <tr>

            <td>@s.StudentId</td>

            <td>
               <input @bind="s.StudentName" />
            </td>

            <td>
               <input @bind="s.Email" />
            </td>

            <td>
               <input @bind="s.DepartmentId"
                      type="number" />
            </td>

            <td>
```

```
                <button @onclick="() => UpdateStudent(s)">
                    Update
                </button>

                <button @onclick="() => DeleteStudent(s.StudentId)">
                    Delete
                </button>

            </td>

        </tr>
    }

</tbody>

</table>


@code
{
    List<Student> students = new();

    Student student = new();


    protected override async Task OnInitializedAsync()
    {
        await LoadStudents();
    }


    async Task LoadStudents()
    {
        var client =
            ClientFactory.CreateClient("CollegeAPI");

        students =
            await client.GetFromJsonAsync<List<Student>>(
                "api/Student") ?? new();
    }


    async Task AddStudent()
    {
        var client =
            ClientFactory.CreateClient("CollegeAPI");

        await client.PostAsJsonAsync(
            "api/Student",
            student);
```

```
        student = new();

        await LoadStudents();
    }


    async Task UpdateStudent(Student s)
    {
        var client =
            ClientFactory.CreateClient("CollegeAPI");

        await client.PutAsJsonAsync(
            $"api/Student/{s.StudentId}",
            s);

        await LoadStudents();
    }


    async Task DeleteStudent(int id)
    {
        var client =
            ClientFactory.CreateClient("CollegeAPI");

        await client.DeleteAsync(
            $"api/Student/{id}");

        await LoadStudents();
    }


    public class Student
    {
        public int StudentId { get; set; }

        public string StudentName { get; set; } = "";

        public string Email { get; set; } = "";

        public int DepartmentId { get; set; }
    }
}
```